Window Functions in SQL

Detailed Study Notes - Session 36

Aggregate windows - PARTITION BY - ORDER BY - Ranking - Value functions - Frames - LAG and LEAD - Cumulative metrics - Percentiles - Worked problems - Interview questions

Concept notes with worked tables. No code listings.

Contents

1. Introduction to Window Functions 5

1.1 The problem window functions solve . . . 5

1.2 Visualising the idea . . . 5

1.3 Where window functions are used . . . 6

1.4 Anatomy of a window function . . . 6

1.5 Categories of window functions . . . 7

1.6 The datasets used in these notes . . . 7

2. Aggregate Functions with OVER 12

2.1 An empty OVER clause: the whole table is the window . . . 12

2.2 Adding PARTITION BY: one window per group . . . 12

2.3 Using the partition value for comparison . . . 13

2.4 Filtering on a window value: students above their branch average . . . 14

2.5 Percentage of total . . . 15

2.6 COUNT as a window function . . . 16

3. PARTITION BY and ORDER BY in Depth 18

3.1 How partitioning works internally . . . 18

3.2 Partitioning by more than one column . . . 18

3.3 What ORDER BY does inside a window . . . 19

3.4 The default frame and the problem of ties . . . 19

3.5 Partition and order together . . . 20

4. Ranking Functions 22

4.1 RANK . . . 22

4.2 DENSE_RANK . . . 23

4.3 ROW_NUMBER . . . 24

4.4 Top-N per group . . . 25

4.5 The Nth highest value per group . . . 26

4.6 Removing duplicate rows . . . 27

4.7 Order sequence per customer: first, second, third order . . . 28

5. Value Functions: FIRST_VALUE, LAST_VALUE, NTH_VALUE 29

5.1 FIRST_VALUE . . . 29

5.2 LAST_VALUE and the frame trap . . . 30

5.3 NTH_VALUE . . . 31

6. Window Frames **34**

6.1 Frame boundaries . . . 34

6.2 ROWS mode and RANGE mode . . . 34

6.3 A catalogue of useful frames . . . 34

6.4 Frames illustrated on the temperature data . . . 35

6.5 Moving averages . . . 36

7. LAG and LEAD **38**

7.1 Arguments . . . 38

7.2 LAG: the previous value . . . 38

7.3 LEAD: the next value . . . 39

7.4 Larger offsets . . . 39

7.5 Detecting streaks and changes of direction . . . 40

7.6 Batting: runs compared with the previous innings . . . 41

8. Cumulative Calculations **42**

8.1 Cumulative sum . . . 42

8.2 When was a milestone reached? . . . 42

8.3 Cumulative sum per partition . . . 42

8.4 Running percentage of total (Pareto analysis) . . . 43

8.5 Running count and running distinct measures . . . 44

9. Distribution Functions: NTILE, PERCENT_RANK, CUME_DIST, Percentiles **45**

9.1 NTILE: splitting rows into buckets . . . 45

9.2 PERCENT_RANK . . . 46

9.3 CUME_DIST . . . 46

9.4 Percentiles and the median . . . 47

10. Named Windows and Writing Clean Window Queries **49**

10.1 The WINDOW clause . . . 49

10.2 Readability guidelines . . . 49

10.3 Performance notes . . . 49

11. Worked Problems **50**

Problem 1 - Students who topped their branch . . . 50

Problem 2 - Each branch's top scorer and lowest scorer on one row per branch . . . 50

Problem 3 - Monthly revenue with running total and month-over-month growth . . . 50

Problem 4 - Each customer's favourite restaurant . . . 51

Problem 5 - Restaurants ranked by revenue each month . . . 51

Problem 6 - Employees earning more than their department average 51
Problem 7 - Batters with three consecutive scores of 40 or more 52
Problem 8 - Best and worst match of every batter 52
Problem 9 - Share of each batter in the team's runs 53
Problem 10 - Customers whose order value increased every time 53

12. Common Mistakes and How to Avoid Them **55**

13. Interview Questions with Answers **57**

14. Practice Exercises **60**

Expected results 60

15. Quick Revision Sheet **63**

15.1 Function reference 63
15.2 Ten points to remember 63
15.3 Pattern recipes in words 63

16. Glossary **65**

1. Introduction to Window Functions

A **window function** performs a calculation across a set of rows that are somehow related to the current row, and returns the answer *on every row*. The set of related rows is called the **window**. The defining feature is that the rows are not collapsed: if the input has fifteen rows, the output still has fifteen rows, each one now carrying an extra value that was computed by looking at its neighbours.

This single property is what makes window functions so useful in data analysis. Many real questions need two levels of detail at the same time: the detail of an individual record (one student, one order, one match) and the context of a group (the whole class, all of that customer's orders, that player's whole season). Before window functions, answering such questions meant computing the group-level numbers separately and then joining them back to the detailed rows. Window functions let you do both in one pass.

1.1 The problem window functions solve

Think about the ordinary aggregate functions: SUM, AVG, MIN, MAX and COUNT. When you use them with grouping, the database takes all rows that belong to the same group and squeezes them into one output row. If you group students by branch and ask for the average marks, you get one row per branch. The individual student names are gone, because a single output row cannot hold four different names.

Now suppose the question is: *show every student together with the average marks of their branch*. Grouping alone cannot answer this, because grouping throws away the student-level detail. What we want is to keep every student row and simply attach the branch average beside it. That is exactly what a window function does.

> **Key idea:** Grouping reduces many rows to one row per group. A window function keeps every row and adds a computed value to each row, calculated over that row's window.

1.2 Visualising the idea

Imagine the table printed on paper, and imagine a cardboard frame with a rectangular hole cut out of it. You lay the frame on the table so that only certain rows show through the hole. For the current row, you perform a calculation using only the visible rows, write the answer in a new column, then move the frame for the next row. The hole is the window. Sometimes the hole is the size of the whole table (every row sees every other row), sometimes it covers only the rows of the same branch, and sometimes it covers only the rows that come before the current row in some order.

Row being processed	Rows visible through the window	Value written on the row
Aarav (CSE, 91)	All four CSE students	Average of CSE = 86.75
Diya (CSE, 87)	All four CSE students	Average of CSE = 86.75
Rohan (ECE, 84)	All four ECE students	Average of ECE = 82.25
Pooja (MECH, 88)	All four MECH students	Average of MECH = 78.5

Table 1.1 - How a window partitioned by branch behaves for a few rows

1.3 Where window functions are used

Window functions are among the most frequently used tools of a data analyst. Some typical situations are listed below. Every item on this list is covered with a worked example later in these notes.

Business question	Window concept that answers it
Who are the top 3 students in each branch?	Ranking within a partition
What is the running total of revenue through the year?	Cumulative sum with an ordered window
How much did views grow compared with last month?	Looking at the previous row (LAG)
What share of the total does each product contribute?	Aggregate over the whole table
What was each customer's first order?	FIRST_VALUE or ROW_NUMBER within a customer
What is the 3-day moving average temperature?	A sliding frame of rows
In which match did a batter cross 500 career runs?	Running sum plus a filter
Which employees earn above their department average?	Partition average compared with row value
Remove duplicate records, keeping only one copy	ROW_NUMBER within the duplicate key

Table 1.2 - Common analytical questions and the window idea behind each

1.4 Anatomy of a window function

Every window function call is made of two parts: the **function** itself (what to calculate) and the **OVER clause** (which rows to calculate it over). The OVER clause is the signal to the database that this is a window function and not an ordinary aggregate. Inside OVER there can be up to three optional components.

Component	What it controls	If omitted
PARTITION BY	Splits the rows into independent groups; the calculation restarts for every group.	The whole result set is one single partition.
ORDER BY	Sorts the rows inside each partition; needed for ranking, running totals, LAG/LEAD.	Rows have no defined order; every row sees the whole partition.
Frame (ROWS / RANGE BETWEEN)	Narrows the window to a sliding range of rows around the current row.	A default frame is used, which depends on whether ORDER BY is present.

Table 1.3 - The three parts that can appear inside OVER

A useful way to read any window function is as a sentence: *compute THIS FUNCTION, for rows grouped BY THIS COLUMN, arranged IN THIS ORDER, looking at THIS RANGE of rows*. When the OVER clause is completely empty, the sentence becomes *compute this function over all rows*, which means every row receives the same answer.

1.5 Categories of window functions

Window functions fall into three families. These notes treat each family in its own chapter.

Family	Functions	Typical purpose
Aggregate window functions	SUM, AVG, MIN, MAX, COUNT	Group totals and averages on every row, running totals, moving averages
Ranking window functions	ROW_NUMBER, RANK, DENSE_RANK, NTILE, PERCENT_RANK, CUME_DIST	Positions, top-N per group, buckets, percentiles
Value (navigation) functions	LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE	Fetching a value from another row of the window

Table 1.4 - The three families of window functions

> **Note:** Window functions are part of the SQL standard and are supported by MySQL (version 8.0 onwards), PostgreSQL, SQL Server, Oracle, SQLite (3.25 onwards) and all modern cloud warehouses. Older MySQL 5.x does not support them.

1.6 The datasets used in these notes

To keep the explanations concrete, the same few small tables are used throughout. They are small enough to check by hand, which is the best way to build intuition. Every output table in these notes was computed from these inputs, so you can verify each number yourself.

Dataset A - Student marks

Fifteen students from four engineering branches. Some marks are deliberately repeated (two 91s in CSE, two 84s in ECE, two 88s in MECH) so that the behaviour of ranking functions on ties can be seen clearly.

student_id	name	branch	marks
1	Aarav	CSE	91
2	Diya	CSE	87
3	Kabir	CSE	91
4	Meera	CSE	78
5	Rohan	ECE	84
6	Sneha	ECE	92
7	Vikram	ECE	84
8	Ananya	ECE	69
9	Ishaan	MECH	73
10	Pooja	MECH	88
11	Arjun	MECH	65
12	Nisha	MECH	88
13	Rahul	EEE	79
14	Kavya	EEE	95
15	Siddharth	EEE	71

Table 1.5 - Students (student_id, name, branch, marks)

Dataset B - Food delivery orders

Sixteen orders placed by four customers across four restaurants during the first quarter of the year. This table is used for per-customer running totals, first and last orders, and gaps between orders.

order_id	customer	restaurant	order_date	amount
101	Priya	Spice Route	2023-01-04	450
102	Amit	Burger Barn	2023-01-06	320
103	Priya	Burger Barn	2023-01-11	280
104	Karan	Dosa Point	2023-01-15	190
105	Amit	Spice Route	2023-01-19	610
106	Neha	Pizza Hub	2023-01-22	540
107	Priya	Pizza Hub	2023-02-02	720
108	Karan	Spice Route	2023-02-05	380
109	Neha	Dosa Point	2023-02-09	210
110	Amit	Pizza Hub	2023-02-14	460
111	Karan	Burger Barn	2023-02-20	300
112	Priya	Dosa Point	2023-03-01	240
113	Neha	Spice Route	2023-03-07	590
114	Amit	Dosa Point	2023-03-12	170
115	Karan	Pizza Hub	2023-03-18	650
116	Neha	Burger Barn	2023-03-25	350

Table 1.6 - Orders (order_id, customer, restaurant, order_date, amount)

Dataset C - Batting scores

Nine batters from three teams, each with scores from ten matches (90 rows in total). Only the first match of each batter is printed here; the full season totals appear in the chapters that use them.

match_no	batter	team	runs
1	R. Sharma	Mumbai	45
1	S. Yadav	Mumbai	23
1	I. Kishan	Mumbai	30
1	V. Kohli	Bangalore	82
1	F. du Plessis	Bangalore	38
1	G. Maxwell	Bangalore	15
1	S. Gill	Gujarat	104
1	H. Pandya	Gujarat	27
1	D. Miller	Gujarat	62

Table 1.7 - Batting (first match of each batter shown)

team	batter	season_runs
Bangalore	F. du Plessis	468
Bangalore	G. Maxwell	356
Bangalore	V. Kohli	627
Gujarat	D. Miller	414
Gujarat	H. Pandya	391
Gujarat	S. Gill	602
Mumbai	I. Kishan	363
Mumbai	R. Sharma	514
Mumbai	S. Yadav	472

Table 1.8 - Season totals per batter (sum of ten matches)

Dataset D - Monthly channel views, Dataset E - Employees, Dataset F - Daily temperature

month	views
Jan	12000
Feb	15500
Mar	14200
Apr	18900
May	22400
Jun	21000
Jul	26800
Aug	31500
Sep	29700
Oct	35200
Nov	41000
Dec	38600

Table 1.9 - Monthly views of a video channel

emp_id	name	dept	salary
E01	Ravi	Sales	52000
E02	Tara	Sales	61000
E03	Manoj	Sales	61000
E04	Lina	Sales	45000
E05	Gopal	IT	88000
E06	Ritu	IT	97000
E07	Sameer	IT	74000
E08	Fatima	HR	50000
E09	Deepak	HR	58000
E10	Asha	HR	50000

Table 1.10 - Employees (emp_id, name, dept, salary)

day	temp_c
Day 1	31
Day 2	33
Day 3	32
Day 4	35
Day 5	36
Day 6	34
Day 7	30
Day 8	29
Day 9	32
Day 10	34

Table 1.11 - Daily maximum temperature

2. Aggregate Functions with OVER

The simplest way to meet window functions is to take the aggregate functions you already know and add an OVER clause to them. This one change turns them from row-collapsing functions into row-preserving ones.

2.1 An empty OVER clause: the whole table is the window

When the OVER clause is empty, there is no partitioning, no ordering and no frame, so the window for every row is the entire result set. The function is therefore evaluated once over all rows, and the same answer is printed on every row. On its own this looks redundant, but placing the overall figure next to each individual figure is the first step towards comparing them.

name	branch	marks	class_avg	class_max	class_min
Aarav	CSE	91	82.33	95	65
Diya	CSE	87	82.33	95	65
Kabir	CSE	91	82.33	95	65
Meera	CSE	78	82.33	95	65
Rohan	ECE	84	82.33	95	65
Sneha	ECE	92	82.33	95	65
Vikram	ECE	84	82.33	95	65
Ananya	ECE	69	82.33	95	65
Ishaan	MECH	73	82.33	95	65
Pooja	MECH	88	82.33	95	65
Arjun	MECH	65	82.33	95	65
Nisha	MECH	88	82.33	95	65
Rahul	EEE	79	82.33	95	65
Kavya	EEE	95	82.33	95	65
Siddharth	EEE	71	82.33	95	65

Table 2.1 - Average, maximum and minimum over the whole class, repeated on every row

Notice that the output still has fifteen rows and still shows each student's name. The average of all fifteen students is 82.33, the highest mark is 95 and the lowest is 65, and these values appear on every row.

> **Key idea:** An empty OVER() means: treat the entire result as one window. Every row gets the same aggregate value.

2.2 Adding PARTITION BY: one window per group

PARTITION BY divides the rows into groups based on one or more columns. The function is then computed separately for each group, and each row receives the value computed for its own group. It feels similar to grouping, with the crucial difference that the rows are

kept.

Below, the rows are partitioned by branch. Each student now sees only the students of the same branch, so the branch average, the branch topper's mark and the branch head-count are written beside each name.

name	branch	marks	branch_avg	branch_max	branch_min	branch_count
Aarav	CSE	91	86.75	91	78	4
Diya	CSE	87	86.75	91	78	4
Kabir	CSE	91	86.75	91	78	4
Meera	CSE	78	86.75	91	78	4
Rohan	ECE	84	82.25	92	69	4
Sneha	ECE	92	82.25	92	69	4
Vikram	ECE	84	82.25	92	69	4
Ananya	ECE	69	82.25	92	69	4
Rahul	EEE	79	81.67	95	71	3
Kavya	EEE	95	81.67	95	71	3
Siddharth	EEE	71	81.67	95	71	3
Ishaan	MECH	73	78.5	88	65	4
Pooja	MECH	88	78.5	88	65	4
Arjun	MECH	65	78.5	88	65	4
Nisha	MECH	88	78.5	88	65	4

Table 2.2 - Aggregates partitioned by branch

Every CSE row shows 86.75 as the average, every ECE row shows 82.25, and so on. The partitions are completely independent: the calculation for MECH never sees a single CSE row.

2.3 Using the partition value for comparison

Once the group value sits on the same row as the individual value, you can combine them with ordinary arithmetic. This is where the real power begins. Some useful derived columns are:

- **Difference from average**: the student's marks minus the branch average. Positive means above average.
- **Gap to topper**: the branch maximum minus the student's marks. Zero means the student is the topper.
- **Share of branch total**: marks divided by the sum of branch marks, shown as a percentage.
- **Ratio to class average**: marks divided by the overall average, a quick measure of relative performance.

name	branch	marks	branch_avg	diff_from_avg	gap_to_topper	pct_of_branch_total
Aarav	CSE	91	86.75	4.25	0	26.22
Diya	CSE	87	86.75	0.25	4	25.07
Kabir	CSE	91	86.75	4.25	0	26.22
Meera	CSE	78	86.75	-8.75	13	22.48
Rohan	ECE	84	82.25	1.75	8	25.53
Sneha	ECE	92	82.25	9.75	0	27.96
Vikram	ECE	84	82.25	1.75	8	25.53
Ananya	ECE	69	82.25	-13.25	23	20.97
Rahul	EEE	79	81.67	-2.67	16	32.24
Kavya	EEE	95	81.67	13.33	0	38.78
Siddharth	EEE	71	81.67	-10.67	24	28.98
Ishaan	MECH	73	78.5	-5.5	15	23.25
Pooja	MECH	88	78.5	9.5	0	28.03
Arjun	MECH	65	78.5	-13.5	23	20.7
Nisha	MECH	88	78.5	9.5	0	28.03

Table 2.3 - Derived comparison columns built on partition values

2.4 Filtering on a window value: students above their branch average

A very common task is to keep only those rows whose own value is above the value of their group, for example, students who scored more than their branch average. The natural idea is to write the condition directly in the filtering step. That does not work, and the reason is important.

The database evaluates the parts of a query in a fixed logical order. Row filtering happens very early, before any window function has been computed. At the time the filter runs, the branch average simply does not exist yet. Window functions are computed almost at the end, just before the final sorting and limiting.

Step	Logical phase	Can it see window results?
1	FROM and JOIN - gather the source rows	No
2	WHERE - filter individual rows	No
3	GROUP BY - form groups	No
4	HAVING - filter groups	No
5	Window functions are evaluated	They are created here
6	SELECT list is finalised (aliases available)	Yes
7	DISTINCT	Yes
8	ORDER BY - final sort	Yes
9	LIMIT / OFFSET	Yes

Table 2.4 - Logical order of query evaluation

The solution is to do the work in two stages. In the first stage, an inner query computes the window value and gives it a name. In the second stage, an outer query treats that result like an ordinary table and filters on the named column. The inner stage can be written as a subquery in the FROM clause or as a common table expression (a named temporary result declared before the main query). Both are equivalent in meaning.

name	branch	marks	branch_avg
Aarav	CSE	91	86.75
Diya	CSE	87	86.75
Kabir	CSE	91	86.75
Rohan	ECE	84	82.25
Sneha	ECE	92	82.25
Vikram	ECE	84	82.25
Kavya	EEE	95	81.67
Pooja	MECH	88	78.5
Nisha	MECH	88	78.5

Table 2.5 - Students scoring above their own branch average (result of the two-stage approach)

Key idea: You cannot filter on a window function in the same query level where it is computed. Compute it in an inner query (subquery or CTE), then filter on it in the outer query.

2.5 Percentage of total

Share-of-total is one of the most requested metrics in reporting. It answers the question *what fraction of the whole does this row represent?* The recipe is: take the row value, divide by the sum over the appropriate window, multiply by one hundred. The appropriate window depends on what 'the whole' means.

- If 'the whole' is everything, use an empty window: each order's share of total company revenue.

- If 'the whole' is a group, partition by that group: each order's share of that customer's total spend.
- If you need both, compute both columns side by side in the same query.

order_id	customer	restaurant	amount	pct_of_all	cust_total	pct_of_customer
102	Amit	Burger Barn	320	4.95	1560	20.51
105	Amit	Spice Route	610	9.44	1560	39.1
110	Amit	Pizza Hub	460	7.12	1560	29.49
114	Amit	Dosa Point	170	2.63	1560	10.9
104	Karan	Dosa Point	190	2.94	1520	12.5
108	Karan	Spice Route	380	5.88	1520	25
111	Karan	Burger Barn	300	4.64	1520	19.74
115	Karan	Pizza Hub	650	10.06	1520	42.76
106	Neha	Pizza Hub	540	8.36	1690	31.95
109	Neha	Dosa Point	210	3.25	1690	12.43
113	Neha	Spice Route	590	9.13	1690	34.91
116	Neha	Burger Barn	350	5.42	1690	20.71
101	Priya	Spice Route	450	6.97	1690	26.63
103	Priya	Burger Barn	280	4.33	1690	16.57
107	Priya	Pizza Hub	720	11.15	1690	42.6
112	Priya	Dosa Point	240	3.72	1690	14.2

Table 2.6 - Each order's share of all revenue and of its customer's spend

Total revenue across all sixteen orders is 6,460. The four percentages within each customer add up to 100, and all sixteen overall percentages together also add up to 100 (allowing for rounding).

Restaurant share of revenue

Window functions also work on top of grouped results. First the orders are grouped by restaurant to get revenue; then a window sum over the grouped rows gives the grand total, which is used as the denominator. The window is evaluated after grouping (step 5 comes after step 3), which is why this combination is allowed.

restaurant	revenue	pct_share
Pizza Hub	2370	36.69
Spice Route	2030	31.42
Burger Barn	1250	19.35
Dosa Point	810	12.54

Table 2.7 - Revenue share by restaurant (grouping followed by a window sum)

2.6 COUNT as a window function

COUNT with an OVER clause writes the size of the window on every row. With partitioning, it shows how many rows share the same group. This is handy for spotting customers who ordered only once, branches with few students, or for calculating averages manually. Counting a specific column ignores NULL values, whereas counting all rows includes them; this distinction applies to the window version exactly as it does to the ordinary version.

order_id	customer	restaurant	orders_by_customer	orders_at_restaurant
101	Priya	Spice Route	4	4
102	Amit	Burger Barn	4	4
103	Priya	Burger Barn	4	4
104	Karan	Dosa Point	4	4
105	Amit	Spice Route	4	4
106	Neha	Pizza Hub	4	4
107	Priya	Pizza Hub	4	4
108	Karan	Spice Route	4	4
109	Neha	Dosa Point	4	4
110	Amit	Pizza Hub	4	4

Table 2.8 - Window counts by customer and by restaurant (first ten orders shown)

3. PARTITION BY and ORDER BY in Depth

3.1 How partitioning works internally

Conceptually, the database handles PARTITION BY in three steps. First it looks at the partition column(s) of every row and places rows with identical values into the same bucket. Second, it processes each bucket on its own, applying the function only to the rows in that bucket. Third, it writes the result back on each original row. Because of the first step, databases often sort or hash the rows by the partition columns internally, which is why an index on those columns can make window queries faster on large tables.

- Partitions never overlap: each row belongs to exactly one partition.
- A partition can contain a single row; the function then works on just that row.
- NULL values in the partition column are placed together in one partition of their own.
- The number of output rows is unaffected by partitioning; only the values computed change.

3.2 Partitioning by more than one column

You may list several columns after PARTITION BY. A new partition is then formed for every distinct *combination* of those columns. With orders, partitioning by customer and month gives a separate window for Priya-January, Priya-February, Amit-January and so on. This is how you compute, for example, each customer's monthly spend while still keeping every individual order visible.

customer	month	order_id	amount	cust_month_total	cust_month_orders
Amit	Jan	102	320	930	2
Amit	Jan	105	610	930	2
Amit	Feb	110	460	460	1
Amit	Mar	114	170	170	1
Karan	Jan	104	190	190	1
Karan	Feb	108	380	680	2
Karan	Feb	111	300	680	2
Karan	Mar	115	650	650	1
Neha	Jan	106	540	540	1
Neha	Feb	109	210	210	1
Neha	Mar	113	590	940	2
Neha	Mar	116	350	940	2
Priya	Jan	101	450	730	2
Priya	Jan	103	280	730	2
Priya	Feb	107	720	720	1
Priya	Mar	112	240	240	1

Table 3.1 - Windows formed by the combination of customer and month

In the batting data, partitioning by team gives team-level totals, while partitioning by team and batter gives player-level totals within the team. Because every batter plays for only one team here, the second grouping is the same as partitioning by batter alone, but in real data (players who changed teams) the combination matters.

3.3 What ORDER BY does inside a window

ORDER BY inside the OVER clause sorts the rows *within each partition* before the function is applied. It has nothing to do with the order in which the final result is displayed; that is controlled separately by the ORDER BY at the end of the query. The two can be different, and very often are.

For ranking and navigation functions, ORDER BY is essential because 'first', 'previous' and 'rank' have no meaning without an order. For aggregate functions, adding ORDER BY has a surprising side effect: it changes the default frame so that each row only sees rows from the start of the partition up to itself. The aggregate therefore becomes a **running** aggregate.

month	views	total_all	running_total
Jan	12000	306800	12000
Feb	15500	306800	27500
Mar	14200	306800	41700
Apr	18900	306800	60600
May	22400	306800	83000
Jun	21000	306800	104000
Jul	26800	306800	130800
Aug	31500	306800	162300
Sep	29700	306800	192000
Oct	35200	306800	227200
Nov	41000	306800	268200
Dec	38600	306800	306800

Table 3.2 - Sum over an empty window (total_all) and sum over a window ordered by month (running_total)

> **Key idea:** Adding ORDER BY to an aggregate window turns it into a cumulative (running) calculation, because the default frame becomes 'from the first row of the partition up to the current row'.

3.4 The default frame and the problem of ties

When ORDER BY is present and no frame is written, the standard default frame is **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**. The word RANGE means that the frame ends not at the current row itself but at the last row that has the *same ordering value* as the current row. Rows with equal ordering values are called **peers**, and peers are always included together.

The effect is visible when the ordering column has duplicates. Order the students by marks and take a running count: both students with 91 get the same running count, because each one's frame includes the other.

name	marks	running_count_range	running_sum_range
Kavya	95	1	95
Sneha	92	2	187
Kabir	91	4	369
Aarav	91	4	369
Pooja	88	6	545
Nisha	88	6	545
Diya	87	7	632
Rohan	84	9	800
Vikram	84	9	800
Rahul	79	10	879
Meera	78	11	957
Ishaan	73	12	1030
Siddharth	71	13	1101
Ananya	69	14	1170
Arjun	65	15	1235

Table 3.3 - Running count and sum ordered by marks (descending) with the default RANGE frame; peers share values

Look at Aarav and Kabir (both 91): each shows a running count of 4, not 3 and 4. The same happens for the pairs at 88 and 84. If you want a strictly row-by-row running total, either give a frame explicitly in ROWS mode, or add a unique column (such as an id) to the ordering so that no two rows are peers.

Practical tip: Whenever a running total 'jumps' by two rows at once, check whether the ordering column contains duplicates. Adding a tie-breaking column to the ORDER BY is the simplest fix.

3.5 Partition and order together

Most real analyses combine both: partition to restart the calculation for each group, and order to give the rows a sequence inside the group. A running total of each customer's spending, in date order, restarting for every customer, is the classic example.

customer	order_date	restaurant	amount	order_seq	cust_running_spend
Amit	2023-01-06	Burger Barn	320	1	320
Amit	2023-01-19	Spice Route	610	2	930
Amit	2023-02-14	Pizza Hub	460	3	1390
Amit	2023-03-12	Dosa Point	170	4	1560
Karan	2023-01-15	Dosa Point	190	1	190
Karan	2023-02-05	Spice Route	380	2	570
Karan	2023-02-20	Burger Barn	300	3	870
Karan	2023-03-18	Pizza Hub	650	4	1520
Neha	2023-01-22	Pizza Hub	540	1	540
Neha	2023-02-09	Dosa Point	210	2	750
Neha	2023-03-07	Spice Route	590	3	1340
Neha	2023-03-25	Burger Barn	350	4	1690
Priya	2023-01-04	Spice Route	450	1	450
Priya	2023-01-11	Burger Barn	280	2	730
Priya	2023-02-02	Pizza Hub	720	3	1450
Priya	2023-03-01	Dosa Point	240	4	1690

Table 3.4 - Running spend per customer, restarting at each new customer

The running spend returns to the order amount itself each time a new customer begins, because each customer is a separate partition. The order sequence number is produced with a ranking function, described in the next chapter.

4. Ranking Functions

Ranking functions assign a position number to each row within its partition according to the ORDER BY inside the window. They take no argument (the parentheses are empty) and they require an ORDER BY in the OVER clause. The three most important are ROW_NUMBER, RANK and DENSE_RANK. They differ only in how they treat ties, and each is described on its own below.

4.1 RANK

RANK gives each row a position based on the ordering. Rows with equal ordering values share the same rank. After a group of tied rows, the next rank **skips** ahead by the number of rows in the tie. If two students share first place, both get rank 1, and the next student gets rank 3, because two people are ahead of them. This is the way positions are awarded in most sports competitions.

name	branch	marks	rank_in_class
Kavya	EEE	95	1
Sneha	ECE	92	2
Aarav	CSE	91	3
Kabir	CSE	91	3
Pooja	MECH	88	5
Nisha	MECH	88	5
Diya	CSE	87	7
Rohan	ECE	84	8
Vikram	ECE	84	8
Rahul	EEE	79	10
Meera	CSE	78	11
Ishaan	MECH	73	12
Siddharth	EEE	71	13
Ananya	ECE	69	14
Arjun	MECH	65	15

Table 4.1 - RANK over the whole class, ordered by marks descending

Ranks 3, 5 and 8 are each shared by two students (marks 91, 88 and 84), so the rank numbers 4, 6 and 9 never appear. The largest rank value can never exceed the number of rows; here it is 15.

RANK within each branch

branch	name	marks	rank_in_branch
CSE	Aarav	91	1
CSE	Kabir	91	1
CSE	Diya	87	3
CSE	Meera	78	4
ECE	Sneha	92	1
ECE	Rohan	84	2
ECE	Vikram	84	2
ECE	Ananya	69	4
EEE	Kavya	95	1
EEE	Rahul	79	2
EEE	Siddharth	71	3
MECH	Pooja	88	1
MECH	Nisha	88	1
MECH	Ishaan	73	3
MECH	Arjun	65	4

Table 4.2 - RANK partitioned by branch

In CSE, Aarav and Kabir both have 91 and share rank 1; Diya is ranked 3. In MECH, Pooja and Nisha share rank 1 with 88; Ishaan is ranked 3. In ECE, Rohan and Vikram share rank 2 behind Sneha, and Ananya is ranked 4.

4.2 DENSE_RANK

DENSE_RANK also gives tied rows the same rank, but it never leaves gaps. After a tie, the next distinct value simply receives the next integer. The ranks are therefore 'dense': they run 1, 2, 3 ... without holes, and the largest rank equals the number of *distinct* values in the partition. DENSE_RANK is the right tool whenever the question is about distinct levels, such as 'the second-highest salary' or 'the top three distinct scores'.

branch	name	marks	dense_rank_in_branch
CSE	Aarav	91	1
CSE	Kabir	91	1
CSE	Diya	87	2
CSE	Meera	78	3
ECE	Sneha	92	1
ECE	Rohan	84	2
ECE	Vikram	84	2
ECE	Ananya	69	3
EEE	Kavya	95	1
EEE	Rahul	79	2
EEE	Siddharth	71	3
MECH	Pooja	88	1
MECH	Nisha	88	1
MECH	Ishaan	73	2
MECH	Arjun	65	3

Table 4.3 - DENSE_RANK partitioned by branch

In CSE the ranks are now 1, 1, 2, 3; in MECH 1, 1, 2, 3; in ECE 1, 2, 2, 3. No number is skipped.

4.3 ROW_NUMBER

ROW_NUMBER simply numbers the rows 1, 2, 3 ... in the given order, restarting at 1 for every partition. It never produces ties: even if two rows have the same ordering value, they receive different numbers. Which of the tied rows gets the smaller number is not guaranteed unless the ORDER BY includes a tie-breaking column. For reproducible results, always add a unique column (such as an id or a name) at the end of the ordering.

branch	name	marks	student_id	row_num_in_branch
CSE	Aarav	91	1	1
CSE	Kabir	91	3	2
CSE	Diya	87	2	3
CSE	Meera	78	4	4
ECE	Sneha	92	6	1
ECE	Rohan	84	5	2
ECE	Vikram	84	7	3
ECE	Ananya	69	8	4
EEE	Kavya	95	14	1
EEE	Rahul	79	13	2
EEE	Siddharth	71	15	3
MECH	Pooja	88	10	1
MECH	Nisha	88	12	2
MECH	Ishaan	73	9	3
MECH	Arjun	65	11	4

Table 4.4 - ROW_NUMBER partitioned by branch, ordered by marks descending then student_id

Typical uses of ROW_NUMBER

- **Exactly N rows per group**: when you need precisely one or precisely three rows per group regardless of ties.
- **Removing duplicates**: number the rows within each set of duplicates and keep only row number 1.
- **Latest record per entity**: order by date descending within each customer and keep row number 1.
- **Pagination**: number all rows and fetch numbers 11 to 20 for the second page.
- **Generating identifiers**: create roll numbers or sequence numbers inside a group.

4.4 Top-N per group

'Show the top two students of every branch' is one of the most common analytics and interview questions. The recipe uses the two-stage pattern from Chapter 2.

Stage 1 - in an inner query, compute a rank for each student partitioned by branch and ordered by marks descending. **Stage 2** - in the outer query, keep only rows where that rank is less than or equal to two. Which ranking function you choose decides how ties are handled, so choose it according to the business rule.

branch	name	marks	rank_in_branch
CSE	Aarav	91	1
CSE	Kabir	91	1
ECE	Sneha	92	1
ECE	Rohan	84	2
ECE	Vikram	84	2
EEE	Kavya	95	1
EEE	Rahul	79	2
MECH	Pooja	88	1
MECH	Nisha	88	1

Table 4.5 - Top two per branch using RANK (tied students are all included)

With RANK, ECE returns three students because Rohan and Vikram tie for second place. CSE and MECH return their two joint toppers. If the rule were 'exactly two names per branch', ROW_NUMBER would be used instead; if the rule were 'everyone holding one of the top two distinct scores', DENSE_RANK would be used.

Top three batters of each team by season runs

Here the data is first grouped to get each batter's season total. A ranking window partitioned by team and ordered by total runs is then applied on top of the grouped rows. This 'aggregate first, then rank' pattern is extremely common in reporting.

team	batter	runs	team_rank
Bangalore	V. Kohli	627	1
Bangalore	F. du Plessis	468	2
Bangalore	G. Maxwell	356	3
Gujarat	S. Gill	602	1
Gujarat	D. Miller	414	2
Gujarat	H. Pandya	391	3
Mumbai	R. Sharma	514	1
Mumbai	S. Yadav	472	2
Mumbai	I. Kishan	363	3

Table 4.6 - Batters ranked within their team by season runs

4.5 The Nth highest value per group

To find the second-highest salary in each department, rank salaries within each department using DENSE_RANK (ordered from highest to lowest), then keep rows where the dense rank equals two. DENSE_RANK is used because the question is about the second highest *salary value*, and two people sharing the top salary should not push the real second value out of reach.

emp_id	name	dept	salary	dept_dense_rank
E09	Deepak	HR	58000	1
E08	Fatima	HR	50000	2
E10	Asha	HR	50000	2
E06	Ritu	IT	97000	1
E05	Gopal	IT	88000	2
E07	Sameer	IT	74000	3
E02	Tara	Sales	61000	1
E03	Manoj	Sales	61000	1
E01	Ravi	Sales	52000	2
E04	Lina	Sales	45000	3

Table 4.7 - Salaries dense-ranked within each department

dept	name	salary
HR	Fatima	50000
HR	Asha	50000
IT	Gopal	88000
Sales	Ravi	52000

Table 4.8 - Employees earning the second-highest salary of their department

In Sales, Tara and Manoj share the top salary of 61,000, so the second-highest salary value is Ravi's 52,000. In HR, Fatima and Asha share 50,000, which is the second-highest value after Deepak's 58,000, so both appear.

4.6 Removing duplicate rows

Suppose an order file was loaded twice by mistake, so some orders appear two or three times. To keep one copy of each, partition by the columns that define a duplicate (for example customer, restaurant, date and amount), order by a load timestamp or id, and assign ROW_NUMBER. Every first copy receives 1; every extra copy receives 2, 3 and so on. Deleting or filtering out rows whose number is greater than one leaves a clean table.

load_id	customer	restaurant	order_date	amount	copy_no	action
1	Priya	Spice Route	2023-01-04	450	1	keep
2	Priya	Spice Route	2023-01-04	450	2	remove
3	Amit	Burger Barn	2023-01-06	320	1	keep
4	Karan	Dosa Point	2023-01-15	190	1	keep
5	Karan	Dosa Point	2023-01-15	190	2	remove
6	Karan	Dosa Point	2023-01-15	190	3	remove
7	Neha	Pizza Hub	2023-01-22	540	1	keep

Table 4.9 - Duplicate detection with ROW_NUMBER

4.7 Order sequence per customer: first, second, third order

ROW_NUMBER partitioned by customer and ordered by date labels each customer's orders in sequence. Filtering on the label answers questions such as 'what was each customer's first order?' or 'what did customers spend on their second order?' Retention analysts use this constantly, for example to measure how many customers ever place a second order.

customer	order_no	order_date	restaurant	amount
Amit	1	2023-01-06	Burger Barn	320
Amit	2	2023-01-19	Spice Route	610
Karan	1	2023-01-15	Dosa Point	190
Karan	2	2023-02-05	Spice Route	380
Neha	1	2023-01-22	Pizza Hub	540
Neha	2	2023-02-09	Dosa Point	210
Priya	1	2023-01-04	Spice Route	450
Priya	2	2023-01-11	Burger Barn	280

Table 4.10 - First and second order of every customer

5. Value Functions: FIRST_VALUE, LAST_VALUE, NTH_VALUE

Value functions do not calculate anything new; they *fetch* the value of some column from a particular row of the window and write it on the current row. FIRST_VALUE fetches from the first row of the window, LAST_VALUE from the last row, and NTH_VALUE from the row at a position you specify. All three need an ordered window, since 'first' and 'last' only make sense once the rows are arranged in some order.

5.1 FIRST_VALUE

FIRST_VALUE takes one argument, the column whose value you want. With the window partitioned by branch and ordered by marks descending, the first row of each window is the branch topper, so FIRST_VALUE of name gives the topper's name on every row of that branch. Because the default frame always starts at the first row of the partition, FIRST_VALUE works correctly without any frame being specified.

branch	name	marks	branch_topper	topper_marks
CSE	Aarav	91	Aarav	91
CSE	Kabir	91	Aarav	91
CSE	Diya	87	Aarav	91
CSE	Meera	78	Aarav	91
ECE	Sneha	92	Sneha	92
ECE	Rohan	84	Sneha	92
ECE	Vikram	84	Sneha	92
ECE	Ananya	69	Sneha	92
EEE	Kavya	95	Kavya	95
EEE	Rahul	79	Kavya	95
EEE	Siddharth	71	Kavya	95
MECH	Pooja	88	Pooja	88
MECH	Nisha	88	Pooja	88
MECH	Ishaan	73	Pooja	88
MECH	Arjun	65	Pooja	88

Table 5.1 - FIRST_VALUE of name and marks within each branch (ordered by marks descending, then id)

In CSE and MECH two students share the top mark, so the tie-breaker (student_id) decides whose name is returned. Once again, adding a unique column to the ordering keeps the result predictable.

First order of every customer

customer	order_date	restaurant	first_restaurant	first_order_date
Amit	2023-01-06	Burger Barn	Burger Barn	2023-01-06
Amit	2023-01-19	Spice Route	Burger Barn	2023-01-06
Amit	2023-02-14	Pizza Hub	Burger Barn	2023-01-06
Amit	2023-03-12	Dosa Point	Burger Barn	2023-01-06
Karan	2023-01-15	Dosa Point	Dosa Point	2023-01-15
Karan	2023-02-05	Spice Route	Dosa Point	2023-01-15
Karan	2023-02-20	Burger Barn	Dosa Point	2023-01-15
Karan	2023-03-18	Pizza Hub	Dosa Point	2023-01-15
Neha	2023-01-22	Pizza Hub	Pizza Hub	2023-01-22
Neha	2023-02-09	Dosa Point	Pizza Hub	2023-01-22
Neha	2023-03-07	Spice Route	Pizza Hub	2023-01-22
Neha	2023-03-25	Burger Barn	Pizza Hub	2023-01-22
Priya	2023-01-04	Spice Route	Spice Route	2023-01-04
Priya	2023-01-11	Burger Barn	Spice Route	2023-01-04
Priya	2023-02-02	Pizza Hub	Spice Route	2023-01-04
Priya	2023-03-01	Dosa Point	Spice Route	2023-01-04

Table 5.2 - Each order alongside the customer's first restaurant and first order date

5.2 LAST_VALUE and the frame trap

LAST_VALUE is expected to return the value from the last row of the window. Used naively, however, it seems to return the *current* row's own value, which confuses almost everyone the first time. The cause is the default frame.

When an ORDER BY is present, the default frame runs from the start of the partition up to the current row (and its peers). So for any row, the 'last row of the frame' is the current row itself. LAST_VALUE is working correctly; it is the frame that is too short.

branch	name	marks	last_value_default_frame	last_value_full_frame
CSE	Aarav	91	Kabir	Meera
CSE	Kabir	91	Kabir	Meera
CSE	Diya	87	Diya	Meera
CSE	Meera	78	Meera	Meera
ECE	Sneha	92	Sneha	Ananya
ECE	Rohan	84	Vikram	Ananya
ECE	Vikram	84	Vikram	Ananya
ECE	Ananya	69	Ananya	Ananya
EEE	Kavya	95	Kavya	Siddharth
EEE	Rahul	79	Rahul	Siddharth
EEE	Siddharth	71	Siddharth	Siddharth
MECH	Pooja	88	Nisha	Arjun
MECH	Nisha	88	Nisha	Arjun
MECH	Ishaan	73	Ishaan	Arjun
MECH	Arjun	65	Arjun	Arjun

Table 5.3 - LAST_VALUE of name with the default frame and with a frame covering the whole partition

With the default frame, each row mostly returns itself (tied peers return the last peer). The fix is to widen the frame so that it runs from the first row of the partition to the last row of the partition: **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**. With this frame, every row sees the entire partition, and LAST_VALUE returns the lowest scorer's name for every row in the branch.

> **Key idea:** LAST_VALUE and NTH_VALUE almost always need an explicit frame covering the whole partition. Without it, the frame ends at the current row and the result looks wrong.

5.3 NTH_VALUE

NTH_VALUE takes two arguments: the column and the position N. It returns the value from the Nth row of the frame. If the frame contains fewer than N rows, the result is NULL. As with LAST_VALUE, the default frame causes early rows to return NULL, because when the frame covers only rows 1 and 2, there is no third row yet.

branch	name	marks	second_default	second_full
CSE	Aarav	91	Kabir	Kabir
CSE	Kabir	91	Kabir	Kabir
CSE	Diya	87	Kabir	Kabir
CSE	Meera	78	Kabir	Kabir
ECE	Sneha	92	NULL	Rohan
ECE	Rohan	84	Rohan	Rohan
ECE	Vikram	84	Rohan	Rohan
ECE	Ananya	69	Rohan	Rohan
EEE	Kavya	95	NULL	Rahul
EEE	Rahul	79	Rahul	Rahul
EEE	Siddharth	71	Rahul	Rahul
MECH	Pooja	88	Nisha	Nisha
MECH	Nisha	88	Nisha	Nisha
MECH	Ishaan	73	Nisha	Nisha
MECH	Arjun	65	Nisha	Nisha

Table 5.4 - NTH_VALUE(name, 2) with default frame and with the full-partition frame

With the default frame, Sneha (ECE) and Kavya (EEE) receive NULL because, at their row, the frame contains only one row. In CSE and MECH the topper is tied with another student, so the frame already includes two peers and the second name appears straight away. With the full-partition frame, every row receives the runner-up's name.

Useful applications of value functions

- The price on the first and last day of each month, placed on every daily row.
- Each customer's first and most recent restaurant, for loyalty analysis.
- The difference between each score and the group's best score (current value minus FIRST_VALUE).
- The runner-up of each competition using NTH_VALUE with position 2.
- Carrying a baseline value (the first reading) forward to compute change since the start.

match_no	runs	first_match_runs	change_since_first	season_best
1	82	82	0	113
2	49	82	-33	113
3	113	82	31	113
4	7	82	-75	113
5	64	82	-18	113
6	58	82	-24	113
7	92	82	10	113
8	31	82	-51	113
9	76	82	-6	113
10	55	82	-27	113

Table 5.5 - One batter's scores with a FIRST_VALUE baseline and a full-window MAX

6. Window Frames

A **frame** is a further restriction inside the partition. While the partition decides which group of rows a row belongs to, the frame decides which rows of that group are actually visible when computing the current row's value. Frames move with the current row, which is why they are sometimes called 'sliding windows'. A frame only makes sense when ORDER BY is present, since 'before' and 'after' need an order.

6.1 Frame boundaries

A frame is described by a start boundary and an end boundary. The vocabulary is small and worth memorising.

Boundary	Meaning
UNBOUNDED PRECEDING	The first row of the partition
n PRECEDING	The row n positions before the current row
CURRENT ROW	The current row (in RANGE mode: the current row and its peers)
n FOLLOWING	The row n positions after the current row
UNBOUNDED FOLLOWING	The last row of the partition

Table 6.1 - Frame boundary keywords

The general form reads: *ROWS BETWEEN start AND end*. The start must not come after the end; for example, a frame from 1 FOLLOWING to 1 PRECEDING is invalid. If only a start is given, the end is taken to be the current row.

6.2 ROWS mode and RANGE mode

In **ROWS** mode, the boundaries count physical rows: '2 PRECEDING' means exactly the two rows immediately before. In **RANGE** mode, boundaries are defined by the values of the ordering column: rows are included if their ordering value lies within a certain distance of the current row's value, and all peers are treated as a unit. RANGE with numeric offsets (for example, 'all rows whose date is within 7 days before the current date') is supported by PostgreSQL, Oracle and MySQL 8, while ROWS mode is available everywhere and is the one used most in day-to-day analysis.

6.3 A catalogue of useful frames

Frame	Rows visible for the current row	Typical use
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	Start of partition up to this row	Running total, running average, running max
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW	This row and the two before it	3-period trailing moving average
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING	Previous, current and next row	Centred 3-point smoothing
ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING	This row up to the end	Remaining total, 'still to come'
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	The whole partition	LAST_VALUE, NTH_VALUE, full-group values
ROWS BETWEEN 1 PRECEDING AND 1 PRECEDING	Only the previous row	Same effect as looking back one row
ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING	All earlier rows, excluding this one	Best score before today, prior total

Table 6.2 - Frequently used frame definitions

6.4 Frames illustrated on the temperature data

The table below applies several frames to the same ten daily temperatures. Read across a row to see how the choice of frame changes the answer for the same day.

day	temp_c	running_max	avg_prev2_cur	avg_prev_cur_next	sum_cur_to_end	max_before_today
Day 1	31	31	31	32	326	NULL
Day 2	33	33	32	32	295	31
Day 3	32	33	32	33.33	262	33
Day 4	35	35	33.33	34.33	230	33
Day 5	36	36	34.33	35	195	35
Day 6	34	36	35	33.33	159	36
Day 7	30	36	33.33	31	125	36
Day 8	29	36	31	30.33	95	36
Day 9	32	36	30.33	31.67	66	36
Day 10	34	36	31.67	33	34	36

Table 6.3 - Different frames on the same series

- **running_max** - frame from the first day to today; the record high so far.
- **avg_prev2_cur** - frame of today and the two previous days; the first two days average over fewer rows because there are not yet two earlier days.
- **avg_prev_cur_next** - a centred frame; the first and last days have only one neighbour.
- **sum_cur_to_end** - frame from today to the last day; totals what remains.

- **max_before_today** - frame of all earlier days excluding today; Day 1 is NULL because no earlier day exists. A day is a new record if its temperature exceeds this value.

6.5 Moving averages

A moving (rolling) average smooths short-term ups and downs so that the trend is easier to see. With a trailing frame of N rows, the value on each row is the average of that row and the N-1 rows before it. Analysts use 7-day moving averages for daily data (to remove the weekly cycle) and 3-month or 12-month averages for monthly data.

month	views	ma_3_month	ma_3_complete
Jan	12000	12,000	NULL
Feb	15500	13,750	NULL
Mar	14200	13,900	13,900
Apr	18900	16,200	16,200
May	22400	18,500	18,500
Jun	21000	20,767	20,767
Jul	26800	23,400	23,400
Aug	31500	26,433	26,433
Sep	29700	29,333	29,333
Oct	35200	32,133	32,133
Nov	41000	35,300	35,300
Dec	38600	38,267	38,267

Table 6.4 - 3-month moving average of channel views

The column ma_3_month averages whatever rows are available, so January is just its own value. Sometimes it is better to show NULL until a full window is available (ma_3_complete), so that the first values are not misleading. This can be done by also counting the rows in the same frame and showing the average only when the count reaches three.

> **Try it:** Moving averages of batting scores are a nice exercise: a 3-match moving average of runs shows a player's current form better than the season average does.

match_no	runs	form_3_match	season_avg_so_far
1	104	104	104
2	39	71.5	71.5
3	63	68.67	68.67
4	88	63.33	73.5
5	12	54.33	61.2
6	94	64.67	66.67
7	57	54.33	65.29
8	45	65.33	62.75
9	71	57.67	63.67
10	29	48.33	60.2

Table 6.5 - 3-match form and cumulative average for one batter

7. LAG and LEAD

LAG and LEAD fetch a value from another row at a fixed distance from the current row, within the same partition and according to the window's order. LAG looks **backwards** (to earlier rows), LEAD looks **forwards** (to later rows). They are the workhorses of period-over-period analysis.

7.1 Arguments

Argument	Meaning	Default
1. expression	The column (or expression) whose value should be fetched	Required
2. offset	How many rows back (LAG) or forward (LEAD) to look	1
3. default	Value returned when the target row does not exist	NULL

Table 7.1 - Arguments of LAG and LEAD

LAG and LEAD do not accept a frame; they always count rows from the current row inside the partition. They do need an ORDER BY in the window, otherwise 'previous' has no meaning.

7.2 LAG: the previous value

month	views	prev_month_views	change	pct_change
Jan	12000	NULL	NULL	NULL
Feb	15500	12,000	3,500	29.17
Mar	14200	15,500	-1,300	-8.39
Apr	18900	14,200	4,700	33.1
May	22400	18,900	3,500	18.52
Jun	21000	22,400	-1,400	-6.25
Jul	26800	21,000	5,800	27.62
Aug	31500	26,800	4,700	17.54
Sep	29700	31,500	-1,800	-5.71
Oct	35200	29,700	5,500	18.52
Nov	41000	35,200	5,800	16.48
Dec	38600	41,000	-2,400	-5.85

Table 7.2 - Month-over-month change using LAG(views, 1)

January has no earlier month, so LAG returns NULL, and any arithmetic involving NULL is also NULL. The percentage change formula is: (current minus previous) divided by previous, multiplied by 100. Negative values (March, June, September, December) show months where views dropped.

Note: If you would rather show 0 than NULL for the first row, supply 0 as the third argument of LAG. Be careful with percentage change though: dividing by a default of 0 would cause a division error, so NULL is usually safer.

7.3 LEAD: the next value

LEAD mirrors LAG. On the orders table, LEAD of order_date partitioned by customer and ordered by date gives the date of the customer's next order. Subtracting the current date from it gives the number of days until the customer came back, a key measure in customer retention analysis.

customer	order_date	amount	prev_amount	next_order_date	days_to_next
Amit	2023-01-06	320	NULL	2023-01-19	13
Amit	2023-01-19	610	320	2023-02-14	26
Amit	2023-02-14	460	610	2023-03-12	26
Amit	2023-03-12	170	460	NULL	NULL
Karan	2023-01-15	190	NULL	2023-02-05	21
Karan	2023-02-05	380	190	2023-02-20	15
Karan	2023-02-20	300	380	2023-03-18	26
Karan	2023-03-18	650	300	NULL	NULL
Neha	2023-01-22	540	NULL	2023-02-09	18
Neha	2023-02-09	210	540	2023-03-07	26
Neha	2023-03-07	590	210	2023-03-25	18
Neha	2023-03-25	350	590	NULL	NULL
Priya	2023-01-04	450	NULL	2023-01-11	7
Priya	2023-01-11	280	450	2023-02-02	22
Priya	2023-02-02	720	280	2023-03-01	27
Priya	2023-03-01	240	720	NULL	NULL

Table 7.3 - LAG of amount and LEAD of order date within each customer

Each customer's last order has NULL as the next date, because there is no later order in that partition. Averaging days_to_next per customer gives the typical re-order interval:

customer	avg_days_between_orders
Amit	21.7
Karan	20.7
Neha	20.7
Priya	18.7

Table 7.4 - Average days between orders per customer (aggregate over the LEAD result)

7.4 Larger offsets

The offset does not have to be 1. With monthly data, LAG with an offset of 12 fetches the same month of the previous year (year-over-year comparison), and an offset of 3 fetches the value from the previous quarter. With the batting data, LAG with an offset of 2 fetches the score from two matches ago.

month	views	views_3_months_ago	growth_over_quarter_pct
Jan	12000	NULL	NULL
Feb	15500	NULL	NULL
Mar	14200	NULL	NULL
Apr	18900	12,000	57.5
May	22400	15,500	44.52
Jun	21000	14,200	47.89
Jul	26800	18,900	41.8
Aug	31500	22,400	40.62
Sep	29700	21,000	41.43
Oct	35200	26,800	31.34
Nov	41000	31,500	30.16
Dec	38600	29,700	29.97

Table 7.5 - LAG with offset 3

7.5 Detecting streaks and changes of direction

Comparing each row with the previous one lets you label every row as 'up', 'down' or 'same'. From those labels, streaks of consecutive rises or falls can be found. The temperature example below marks each day relative to the day before and flags days that set a new running high.

day	temp_c	prev_day	direction	next_day	new_record
Day 1	31	NULL	-	33	yes
Day 2	33	31	up	32	yes
Day 3	32	33	down	35	no
Day 4	35	32	up	36	yes
Day 5	36	35	up	34	yes
Day 6	34	36	down	30	no
Day 7	30	34	down	29	no
Day 8	29	30	down	32	no
Day 9	32	29	up	34	no
Day 10	34	32	up	NULL	no

Table 7.6 - Direction of change (via LAG) and new records

Days 6, 7 and 8 form a streak of three consecutive drops. Days 1, 2, 4 and 5 each set a new high relative to all earlier days. A streak can be measured exactly by combining LAG

with a running count: every time the direction changes, a new streak number begins, and rows with the same streak number are counted together.

7.6 Batting: runs compared with the previous innings

match_no	runs	prev_innings	next_innings	better_than_prev
1	45	NULL	12	-
2	12	45	78	no
3	78	12	33	yes
4	33	78	101	no
5	101	33	8	yes
6	8	101	56	no
7	56	8	64	yes
8	64	56	27	yes
9	27	64	90	no
10	90	27	NULL	yes

Table 7.7 - LAG and LEAD on one batter's innings

8. Cumulative Calculations

Cumulative (running) calculations accumulate a value from the beginning of an ordered partition up to the current row. They show how a quantity builds up over time, and they answer 'when did we reach X?' questions.

8.1 Cumulative sum

A cumulative sum uses SUM with an ordered window and the frame from the first row to the current row. For a clean row-by-row total, write the frame in ROWS mode explicitly (or make sure the ordering column has no duplicates).

match_no	runs	innings	career_runs	career_avg	highest_so_far
1	82	1	82	82	82
2	49	2	131	65.5	82
3	113	3	244	81.33	113
4	7	4	251	62.75	113
5	64	5	315	63	113
6	58	6	373	62.17	113
7	92	7	465	66.43	113
8	31	8	496	62	113
9	76	9	572	63.56	113
10	55	10	627	62.7	113

Table 8.1 - Running runs, running average and running highest score for one batter

8.2 When was a milestone reached?

To find the match in which a batter crossed 500 runs, compute the running total in an inner query, then in the outer query keep the rows where the running total is at least 500 and take the earliest such match per batter. An elegant variant uses LAG on the running total: the milestone match is the one where the running total is at least 500 but the previous running total was below 500.

batter	team	match_no	prev_running	running
R. Sharma	Mumbai	10	424	514
S. Gill	Gujarat	8	457	502
V. Kohli	Bangalore	9	496	572

Table 8.2 - Match in which each batter crossed 500 season runs

Batters who never reach 500 in the ten matches simply do not appear. In this data they are: D. Miller, F. du Plessis, G. Maxwell, H. Pandya, I. Kishan, S. Yadav.

8.3 Cumulative sum per partition

Adding PARTITION BY restarts the accumulation for each group. The table shows team-level running totals of runs by match: all three batters' scores for a match are added to the team's running total.

team	match_no	team_runs	team_running_total
Bangalore	1	135	135
Bangalore	2	179	314
Bangalore	3	142	456
Bangalore	4	144	600
Bangalore	5	136	736
Gujarat	1	193	193
Gujarat	2	108	301
Gujarat	3	130	431
Gujarat	4	162	593
Gujarat	5	158	751
Mumbai	1	98	98
Mumbai	2	84	182
Mumbai	3	177	359
Mumbai	4	143	502
Mumbai	5	163	665

Table 8.3 - Team runs per match and running team total (first five matches per team)

8.4 Running percentage of total (Pareto analysis)

Dividing the running total by the grand total gives the cumulative percentage. Ordering the items from largest to smallest and looking at the cumulative percentage reveals which few items produce most of the total, the idea behind the famous 80/20 rule. Below, the season totals of all nine batters are ordered from highest to lowest.

batter	runs	running_runs	cum_pct	pct_of_total
V. Kohli	627	627	14.9	14.9
S. Gill	602	1229	29.21	14.31
R. Sharma	514	1743	41.43	12.22
S. Yadav	472	2215	52.65	11.22
F. du Plessis	468	2683	63.77	11.12
D. Miller	414	3097	73.62	9.84
H. Pandya	391	3488	82.91	9.29
I. Kishan	363	3851	91.54	8.63
G. Maxwell	356	4207	100	8.46

Table 8.4 - Cumulative share of all runs, batters ordered by season runs

8.5 Running count and running distinct measures

COUNT over an ordered window gives the number of rows so far, the same as ROW_NUMBER when there are no ties. Running minimum and maximum track the lowest and highest values seen so far. Counting distinct values inside a window is not supported by most databases; a common workaround is to rank values with DENSE_RANK and take the running maximum of that rank.

order_date	customer	amount	orders_so_far	revenue_so_far	biggest_order_so_far	customers_so_far
2023-01-04	Priya	450	1	450	450	1
2023-01-06	Amit	320	2	770	450	2
2023-01-11	Priya	280	3	1050	450	2
2023-01-15	Karan	190	4	1240	450	3
2023-01-19	Amit	610	5	1850	610	3
2023-01-22	Neha	540	6	2390	610	4
2023-02-02	Priya	720	7	3110	720	4
2023-02-05	Karan	380	8	3490	720	4
2023-02-09	Neha	210	9	3700	720	4
2023-02-14	Amit	460	10	4160	720	4
2023-02-20	Karan	300	11	4460	720	4
2023-03-01	Priya	240	12	4700	720	4
2023-03-07	Neha	590	13	5290	720	4
2023-03-12	Amit	170	14	5460	720	4
2023-03-18	Karan	650	15	6110	720	4
2023-03-25	Neha	350	16	6460	720	4

Table 8.5 - Company-wide running measures in date order

9. Distribution Functions: NTILE, PERCENT_RANK, CUME_DIST, Percentiles

Distribution functions describe where a row sits within the spread of its partition, and they are the basis of segmentation (splitting customers into tiers) and percentile reporting.

9.1 NTILE: splitting rows into buckets

NTILE(n) divides the ordered rows of each partition into n groups that are as equal in size as possible, and returns the group number (1 to n) for each row. If the rows cannot be split evenly, the earlier groups get one extra row. With 15 students and NTILE(4), the bucket sizes are 4, 4, 4 and 3.

name	branch	marks	quartile	segment
Kavya	EEE	95	1	Top
Sneha	ECE	92	1	Top
Aarav	CSE	91	1	Top
Kabir	CSE	91	1	Top
Pooja	MECH	88	2	Upper middle
Nisha	MECH	88	2	Upper middle
Diya	CSE	87	2	Upper middle
Rohan	ECE	84	2	Upper middle
Vikram	ECE	84	3	Lower middle
Rahul	EEE	79	3	Lower middle
Meera	CSE	78	3	Lower middle
Ishaan	MECH	73	3	Lower middle
Siddharth	EEE	71	4	Bottom
Ananya	ECE	69	4	Bottom
Arjun	MECH	65	4	Bottom

Table 9.1 - NTILE(4) over marks (descending)

Because NTILE fills buckets by position, tied students can end up in different buckets (look at the edges between buckets). A common business use is RFM segmentation, where customers are split into five groups by recency, frequency and monetary value.

Customer tiers from total spend

customer	amount	tier	tier_name
Priya	1690	1	Gold
Neha	1690	1	Gold
Amit	1560	2	Silver
Karan	1520	2	Silver

Table 9.2 - Customers split into two tiers with NTILE(2) over total spend

9.2 PERCENT_RANK

PERCENT_RANK expresses the rank as a fraction between 0 and 1 using the formula (rank minus 1) divided by (number of rows minus 1). The first row always gets 0 and the last distinct value gets 1. It answers the question 'what fraction of the other rows are ranked ahead of this one?'

name	marks	rank_asc	percent_rank
Arjun	65	1	0
Ananya	69	2	0.07
Siddharth	71	3	0.14
Ishaan	73	4	0.21
Meera	78	5	0.29
Rahul	79	6	0.36
Rohan	84	7	0.43
Vikram	84	7	0.43
Diya	87	9	0.57
Pooja	88	10	0.64
Nisha	88	10	0.64
Aarav	91	12	0.79
Kabir	91	12	0.79
Sneha	92	14	0.93
Kavya	95	15	1

Table 9.3 - PERCENT_RANK ordered by marks ascending

Arjun, with the lowest marks, has a percent rank of 0. Kavya, with the highest, has 1. A student with a percent rank of 0.5 scored higher than half of the other students.

9.3 CUME_DIST

CUME_DIST (cumulative distribution) returns the fraction of rows whose ordering value is less than or equal to the current row's value. Its range is greater than 0 and at most 1, and the last row always gets exactly 1. Tied rows receive the same value, the one that counts all of them.

name	marks	cume_dist
Arjun	65	0.07
Ananya	69	0.13
Siddharth	71	0.2
Ishaan	73	0.27
Meera	78	0.33
Rahul	79	0.4
Rohan	84	0.53
Vikram	84	0.53
Diya	87	0.6
Pooja	88	0.73
Nisha	88	0.73
Aarav	91	0.87
Kabir	91	0.87
Sneha	92	0.93
Kavya	95	1

Table 9.4 - CUME_DIST ordered by marks ascending

For example, Rohan and Vikram (84) both get 8/15 = 0.533, because eight of the fifteen students scored 84 or less. Keeping rows with CUME_DIST greater than 0.8 is an easy way to find the top 20 percent.

9.4 Percentiles and the median

A percentile is the value below which a given percentage of observations fall. The median is the 50th percentile. Databases such as PostgreSQL, Oracle and SQL Server provide PERCENTILE_CONT and PERCENTILE_DISC, which take a fraction (0.5 for the median) and an ordering column. PERCENTILE_DISC returns an actual value from the data, while PERCENTILE_CONT interpolates between the two middle values when necessary. In SQL Server they can be used as window functions with PARTITION BY, placing each group's median on every row. MySQL does not provide them, so the median is computed there using ROW_NUMBER and COUNT: the middle row (or the average of the two middle rows) is selected.

branch	students	median_marks
CSE	4	89
ECE	4	84
EEE	3	79
MECH	4	80.5

Table 9.5 - Median marks per branch (continuous median of each partition)

With four students per branch (three in EEE), the median of CSE is the average of 87 and 91, which is 89, while the median of EEE is the middle value 79.

Finding outliers with percentiles

The interquartile range (IQR) method flags values more than 1.5 times the IQR below the first quartile or above the third quartile. Both quartiles can be computed with percentile functions over a window, and each row can then be labelled 'normal' or 'outlier'. This technique is widely used during data cleaning.

Measure	Value (all 90 innings)
First quartile (Q1)	26
Third quartile (Q3)	65.5
IQR = Q3 - Q1	39.5
Lower fence = Q1 - 1.5 x IQR	-33.25
Upper fence = Q3 + 1.5 x IQR	124.75

Table 9.6 - IQR fences for the batting scores

Every innings in this data lies inside the fences, so no score counts as an outlier. In real sales or sensor data, rows outside the fences deserve a closer look.

10. Named Windows and Writing Clean Window Queries

10.1 The WINDOW clause

When several functions in a query use the same window specification, repeating it makes the query long and easy to get wrong. The standard WINDOW clause lets you define the specification once, give it a name, and refer to that name inside each OVER. It is written after HAVING and before ORDER BY. MySQL 8, PostgreSQL and SQLite support it; SQL Server added it in its 2022 version.

A named window can also be extended. For example, a named window that only partitions by branch can be reused by one function as it is and by another function with an extra ORDER BY or frame added on top.

10.2 Readability guidelines

- Give every window column a meaningful alias, such as branch_avg or running_revenue.
- Put the partition and order on separate lines when the specification is long.
- Always write the frame explicitly for running totals, LAST_VALUE and NTH_VALUE, even if the default would work.
- Include a unique tie-breaker in the ORDER BY of ROW_NUMBER, FIRST_VALUE and running totals.
- Use a CTE for the stage that computes window values and a separate outer query for filtering.
- Keep the final ORDER BY separate in your mind from the ORDER BY inside the window.

10.3 Performance notes

- A window function requires the rows to be sorted by the partition and order columns; on big tables this sort is the main cost. An index that matches those columns can remove the sort.
- Several window functions with identical specifications can share a single sort, so reusing the same window is cheaper than using many different ones.
- Filter as early as possible in an inner query to reduce the number of rows that the window has to process, but remember that filtering also changes what is inside the window.
- Frames with UNBOUNDED boundaries are handled efficiently; very wide sliding frames in RANGE mode can be slower.

> **Important:** Filtering before the window and filtering after it produce different results. Removing rows in the inner query means those rows are not part of any window (for example, they no longer count toward a branch average). Decide deliberately which of the two the question needs.

11. Worked Problems

This chapter walks through complete analytical problems. For each one, the approach is described step by step in plain language, followed by the result computed from the sample data.

Problem 1 - Students who topped their branch

Approach: Rank students within each branch by marks descending. Keep rank 1. Using RANK (or DENSE_RANK) keeps joint toppers; ROW_NUMBER would keep just one.

branch	name	marks
CSE	Aarav	91
CSE	Kabir	91
ECE	Sneha	92
EEE	Kavya	95
MECH	Pooja	88
MECH	Nisha	88

Table 11.1 - Branch toppers

Problem 2 - Each branch's top scorer and lowest scorer on one row per branch

Approach: In a window partitioned by branch and ordered by marks descending, with the frame covering the whole partition, take FIRST_VALUE of name and LAST_VALUE of name. Every row of a branch now carries the same pair, so selecting distinct branch, topper and lowest gives one row per branch.

branch	topper	top_marks	lowest	low_marks
CSE	Aarav	91	Meera	78
ECE	Sneha	92	Ananya	69
EEE	Kavya	95	Siddharth	71
MECH	Pooja	88	Arjun	65

Table 11.2 - Topper and lowest scorer per branch

Problem 3 - Monthly revenue with running total and month-over-month growth

Approach: Group orders by month to get monthly revenue. On the grouped rows, compute a running SUM ordered by month, and use LAG to fetch the previous month's revenue. Growth percentage = (current minus previous) divided by previous, times 100.

month	revenue	running_revenue	prev_revenue	growth_pct
January	2390	2390	NULL	NULL
February	2070	4460	2,390	-13.39
March	2000	6460	2,070	-3.38

Table 11.3 - Monthly revenue analysis

Problem 4 - Each customer's favourite restaurant

Approach: Group orders by customer and restaurant, counting orders and summing spend. Rank the restaurants within each customer by spend descending. Keep rank 1.

customer	restaurant	orders	spend
Amit	Spice Route	1	610
Karan	Pizza Hub	1	650
Neha	Spice Route	1	590
Priya	Pizza Hub	1	720

Table 11.4 - Restaurant with the highest spend for each customer

Problem 5 - Restaurants ranked by revenue each month

Approach: Group by month and restaurant to get revenue. Apply DENSE_RANK partitioned by month, ordered by revenue descending. Keep rank 1 to see the monthly winner.

month	restaurant	amount	rank_in_month
Jan	Spice Route	1060	1
Jan	Burger Barn	600	2
Jan	Pizza Hub	540	3
Jan	Dosa Point	190	4
Feb	Pizza Hub	1180	1
Feb	Spice Route	380	2
Feb	Burger Barn	300	3
Feb	Dosa Point	210	4
Mar	Pizza Hub	650	1
Mar	Spice Route	590	2
Mar	Dosa Point	410	3
Mar	Burger Barn	350	4

Table 11.5 - Restaurant revenue ranks by month

Problem 6 - Employees earning more than their department average

Approach: Compute AVG(salary) partitioned by department in a CTE, then keep rows where salary is greater than that average.

name	dept	salary	dept_avg
Ravi	Sales	52000	54,750
Tara	Sales	61000	54,750
Manoj	Sales	61000	54,750
Lina	Sales	45000	54,750
Gopal	IT	88000	86,333.33
Ritu	IT	97000	86,333.33
Sameer	IT	74000	86,333.33
Fatima	HR	50000	52,666.67
Deepak	HR	58000	52,666.67
Asha	HR	50000	52,666.67

Table 11.6 - Salary and department average (all employees)

name	dept	salary	dept_avg
Tara	Sales	61000	54,750
Manoj	Sales	61000	54,750
Gopal	IT	88000	86,333.33
Ritu	IT	97000	86,333.33
Deepak	HR	58000	52,666.67

Table 11.7 - Employees above their department average

Problem 7 - Batters with three consecutive scores of 40 or more

Approach: Order each batter's innings by match number. Use LAG with offsets 1 and 2 to bring the previous two scores onto the current row. A row where all three values are at least 40 ends a qualifying streak.

batter	match_no	two_ago	previous	current
S. Gill	8	94	57	45
S. Gill	9	57	45	71
S. Yadav	4	67	41	88
V. Kohli	3	82	49	113
V. Kohli	7	64	58	92

Table 11.8 - Innings that complete a run of three consecutive 40+ scores

Problem 8 - Best and worst match of every batter

Approach: Rank innings within each batter by runs descending (for best) and ascending (for worst) using ROW_NUMBER with match number as a tie-breaker. Keep rows where either rank equals 1.

batter	best_match	best_runs	worst_match	worst_runs
D. Miller	5	80	2	14
F. du Plessis	7	83	6	9
G. Maxwell	4	77	3	3
H. Pandya	8	70	9	9
I. Kishan	6	81	2	5
R. Sharma	5	101	6	8
S. Gill	1	104	5	12
S. Yadav	4	88	5	15
V. Kohli	3	113	4	7

Table 11.9 - Best and worst innings per batter

Problem 9 - Share of each batter in the team's runs

Approach: After grouping to season totals, divide each batter's runs by SUM of runs partitioned by team.

team	batter	runs	team_total	share_pct
Bangalore	F. du Plessis	468	1451	32.25
Bangalore	G. Maxwell	356	1451	24.53
Bangalore	V. Kohli	627	1451	43.21
Gujarat	D. Miller	414	1407	29.42
Gujarat	H. Pandya	391	1407	27.79
Gujarat	S. Gill	602	1407	42.79
Mumbai	I. Kishan	363	1349	26.91
Mumbai	R. Sharma	514	1349	38.1
Mumbai	S. Yadav	472	1349	34.99

Table 11.10 - Contribution of each batter to team runs

Problem 10 - Customers whose order value increased every time

Approach: Use LAG of amount within each customer. Mark a row as 'increase' when the amount is higher than the previous amount. A customer qualifies if every row after the first is an increase, which can be checked by counting the non-increases per customer in the outer query.

customer	order_date	amount	prev	increase
Amit	2023-01-06	320	NULL	-
Amit	2023-01-19	610	320	yes
Amit	2023-02-14	460	610	no
Amit	2023-03-12	170	460	no
Karan	2023-01-15	190	NULL	-
Karan	2023-02-05	380	190	yes
Karan	2023-02-20	300	380	no
Karan	2023-03-18	650	300	yes
Neha	2023-01-22	540	NULL	-
Neha	2023-02-09	210	540	no
Neha	2023-03-07	590	210	yes
Neha	2023-03-25	350	590	no
Priya	2023-01-04	450	NULL	-
Priya	2023-01-11	280	450	no
Priya	2023-02-02	720	280	yes
Priya	2023-03-01	240	720	no

Table 11.11 - Order value compared with previous order

No customer increased their order value on every single order, so the final result of this problem is empty for the sample data. An empty result is a perfectly valid answer and is worth checking by hand, as here.

12. Common Mistakes and How to Avoid Them

12.1 Filtering on a window function in WHERE

What goes wrong	How to fix it
WHERE runs before window functions exist, so the database reports an error.	Compute the window value in a CTE or subquery, then filter in the outer query.

12.2 Forgetting that ORDER BY changes the frame

What goes wrong	How to fix it
An aggregate that should give the group total instead gives a running total.	Remove ORDER BY from the window when you need the full-group value, or give the full frame explicitly.

12.3 LAST_VALUE returning the current row

What goes wrong	How to fix it
The default frame ends at the current row.	Use a frame from UNBOUNDED PRECEDING to UNBOUNDED FOLLOWING.

12.4 Non-deterministic ROW_NUMBER

What goes wrong	How to fix it
Tied rows get numbers in an arbitrary order that can change between runs.	Add a unique column to the window ORDER BY as a tie-breaker.

12.5 Running totals that jump over ties

What goes wrong	How to fix it
The default RANGE frame treats peers as one unit.	Use ROWS mode or add a tie-breaker to the ordering.

12.6 Choosing the wrong ranking function for top-N

What goes wrong	How to fix it
Too many or too few rows are returned when there are ties.	Decide the business rule for ties first, then pick the function that implements it.

12.7 Mixing up the window ORDER BY and the final ORDER BY

What goes wrong	How to fix it
The output appears in an unexpected order, or the window calculation uses the wrong sequence.	Treat them as two separate decisions; set both explicitly.

12.8 Using window functions in GROUP BY or HAVING

What goes wrong	How to fix it
Window functions are evaluated after grouping, so they cannot be used to form groups.	Compute them in an inner query and group in the outer query.

12.9 Division by NULL or zero in growth formulas

What goes wrong	How to fix it
First rows have NULL previous values; some previous values may be zero.	Leave NULL as NULL, and guard zero denominators with a conditional expression.

12.10 Partitioning by the wrong level

What goes wrong	How to fix it
Totals are computed per customer when they should be per customer and month, or vice versa.	Say the grouping aloud in words before writing it; check a few rows by hand.

13. Interview Questions with Answers

Q1. What is a window function?

Answer: A function that computes a value over a set of rows related to the current row (the window) and returns that value on every row, without collapsing the rows.

Q2. Why can a window function not appear in the WHERE clause?

Answer: Because WHERE is evaluated before window functions in the logical order of query processing. The window value does not yet exist when rows are being filtered.

Q3. What happens if OVER is left empty?

Answer: The whole result set becomes a single window, so the function returns the same value on every row.

Q4. What does PARTITION BY do?

Answer: It divides the rows into independent groups; the window calculation is performed separately within each group and restarts for every new group.

Q5. Is ORDER BY required in a window?

Answer: It is required for ranking functions and LAG/LEAD, and for meaningful FIRST_VALUE/LAST_VALUE/NTH_VALUE. For aggregates it is optional, but adding it produces running aggregates.

Q6. What is the default frame when ORDER BY is present?

Answer: RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, which includes all peers of the current row.

Q7. What is the default frame when ORDER BY is absent?

Answer: The entire partition.

Q8. How are ties handled by RANK?

Answer: Tied rows share a rank, and the following rank is increased by the number of tied rows, leaving gaps.

Q9. How are ties handled by DENSE_RANK?

Answer: Tied rows share a rank, and the next distinct value gets the next consecutive integer, so there are no gaps.

Q10. How are ties handled by ROW_NUMBER?

Answer: Every row gets a distinct number; tied rows are numbered in an order that is arbitrary unless a tie-breaker is given.

Q11. How would you find the second-highest salary in each department?

Answer: DENSE_RANK partitioned by department ordered by salary descending in a CTE, then keep rows with rank 2.

Q12. How would you delete duplicate rows?

Answer: ROW_NUMBER partitioned by the columns that define a duplicate, ordered by an id; remove rows numbered above 1.

Q13. What is the difference in purpose between LAG and LEAD?

Answer: LAG returns a value from a preceding row, LEAD from a following row, at a given offset within the partition.

Q14. What does the third argument of LAG do?

Answer: It supplies a default value to return when the offset reaches beyond the partition, instead of NULL.

Q15. How do you compute a 7-day moving average?

Answer: AVG over a window ordered by date with the frame ROWS BETWEEN 6 PRECEDING AND CURRENT ROW.

Q16. Why might LAST_VALUE return an unexpected result?

Answer: Because the default frame ends at the current row; the frame must be widened to the end of the partition.

Q17. What does NTILE(4) do?

Answer: It splits the ordered rows of each partition into four nearly equal buckets and returns the bucket number.

Q18. What is CUME_DIST?

Answer: The fraction of rows with an ordering value less than or equal to the current row's value.

Q19. What is PERCENT_RANK?

Answer: (rank - 1) divided by (rows in partition - 1); it ranges from 0 to 1.

Q20. Can a window function be used together with GROUP BY?

Answer: Yes. The window is applied to the grouped rows, after aggregation; for example, ranking groups by their totals.

Q21. Can you nest one window function inside another?

Answer: No. Compute the first in an inner query and apply the second in an outer query.

Q22. What is a named window?

Answer: A window specification defined once in a WINDOW clause and referred to by name in several OVER clauses.

Q23. How do you find each customer's latest order?

Answer: ROW_NUMBER partitioned by customer ordered by date descending; keep row number 1.

Q24. How do you find the running total per customer?

Answer: SUM of amount partitioned by customer, ordered by date, with the frame from the first row to the current row.

Q25. How do you compute year-over-year growth on monthly data?

Answer: LAG with an offset of 12 over the months, then (current - previous) / previous x 100.

14. Practice Exercises

Try each exercise on the sample datasets. Describe your approach in words first, then check your answer against the expected result given below the exercise list.

1. For every student, show the difference between their marks and the class topper's marks.
2. Show the top scorer of every branch using ROW_NUMBER, with student_id as tie-breaker.
3. For every order, show the running count of orders placed by that customer.
4. Show the amount of each customer's most recent order next to every one of their orders.
5. Find the month with the largest percentage increase in views.
6. For each batter, find the number of innings with a score above their own season average.
7. Find the restaurant with the most orders in March.
8. Split the nine batters into three equal groups by season runs.
9. For each team, show the gap in season runs between the top batter and every other batter.
10. Find days on which the temperature was higher than on both the previous and the next day.

Expected results

name	marks	gap_to_class_topper
Aarav	91	4
Diya	87	8
Kabir	91	4
Meera	78	17
Rohan	84	11
Sneha	92	3

Exercise 1 (first six rows)

branch	name	marks
CSE	Aarav	91
ECE	Sneha	92
EEE	Kavya	95
MECH	Pooja	88

Exercise 2

customer	order_date	running_orders
Priya	2023-01-04	1
Priya	2023-01-11	2
Priya	2023-02-02	3
Priya	2023-03-01	4

Exercise 3 (Priya's rows)

customer	latest_amount
Amit	170
Karan	650
Neha	350
Priya	240

Exercise 4 (one row per customer shown)

month	views	pct_change
Apr	18900	33.1

Exercise 5

batter	innings_above_avg
D. Miller	5
F. du Plessis	5
G. Maxwell	4
H. Pandya	5
I. Kishan	5
R. Sharma	5
S. Gill	5
S. Yadav	5
V. Kohli	5

Exercise 6

restaurant	march_orders
Burger Barn	1
Dosa Point	2
Pizza Hub	1
Spice Route	1

Exercise 7 (Dosa Point leads March with two orders)

batter	runs	group
V. Kohli	627	1
S. Gill	602	1
R. Sharma	514	1
S. Yadav	472	2
F. du Plessis	468	2
D. Miller	414	2
H. Pandya	391	3
I. Kishan	363	3
G. Maxwell	356	3

Exercise 8

team	batter	runs	gap_to_team_top
Bangalore	F. du Plessis	468	159
Bangalore	G. Maxwell	356	271
Bangalore	V. Kohli	627	0
Gujarat	D. Miller	414	188
Gujarat	H. Pandya	391	211
Gujarat	S. Gill	602	0
Mumbai	I. Kishan	363	151
Mumbai	R. Sharma	514	0
Mumbai	S. Yadav	472	42

Exercise 9

day	temp_c	prev	next
Day 2	33	31	32
Day 5	36	35	34

Exercise 10 (local peaks)

15. Quick Revision Sheet

15.1 Function reference

Function	Arguments	Needs ORDER BY?	Accepts frame?	Returns
SUM / AVG / MIN / MAX / COUNT	column	Optional	Yes	Aggregate over the frame
ROW_NUMBER	none	Yes	No	Unique sequence number
RANK	none	Yes	No	Rank with gaps after ties
DENSE_RANK	none	Yes	No	Rank without gaps
NTILE	number of buckets	Yes	No	Bucket number
PERCENT_RANK	none	Yes	No	Relative rank 0 to 1
CUME_DIST	none	Yes	No	Cumulative fraction, up to 1
LAG	column, offset, default	Yes	No	Value from an earlier row
LEAD	column, offset, default	Yes	No	Value from a later row
FIRST_VALUE	column	Yes	Yes	Value from first row of frame
LAST_VALUE	column	Yes	Yes (widen it)	Value from last row of frame
NTH_VALUE	column, n	Yes	Yes (widen it)	Value from nth row of frame

Table 15.1 - Window function quick reference

15.2 Ten points to remember

1. Window functions keep every row; grouping collapses rows.
2. OVER() with nothing inside means the whole table is the window.
3. PARTITION BY restarts the calculation for every group.
4. ORDER BY inside OVER defines sequence and, for aggregates, creates running totals.
5. Default frame with ORDER BY: from the partition start to the current row and its peers.
6. RANK leaves gaps after ties, DENSE_RANK does not, ROW_NUMBER never ties.
7. LAST_VALUE and NTH_VALUE need the frame widened to the whole partition.
8. LAG looks back, LEAD looks forward; the first or last rows get NULL unless a default is given.
9. Window values cannot be filtered in WHERE; use a CTE or subquery.
10. Always add a tie-breaker when the exact order of tied rows matters.

15.3 Pattern recipes in words

Goal	Recipe
Group value on every row	Aggregate + PARTITION BY group, no ORDER BY
Share of total	Row value / SUM over the relevant window x 100
Running total	SUM + ORDER BY + frame from UNBOUNDED PRECEDING to CURRENT ROW
Moving average of N	AVG + ORDER BY + frame from N-1 PRECEDING to CURRENT ROW
Top N per group	Rank in CTE partitioned by group, filter rank <= N
Nth highest per group	DENSE_RANK in CTE, filter rank = N
Latest record per entity	ROW_NUMBER partitioned by entity, ordered by date descending, keep 1
Remove duplicates	ROW_NUMBER partitioned by the duplicate key, keep 1
Period-over-period change	Current value minus LAG(value)
Days until next event	LEAD(date) minus current date
Milestone crossing	Running total >= target and previous running total < target
Segmentation	NTILE(n) ordered by the metric
Top X percent	CUME_DIST or PERCENT_RANK filtered at the threshold

Table 15.2 - Recipes

16. Glossary

Term	Meaning
Window	The set of rows over which a window function is computed for the current row.
OVER clause	The part of a window function call that defines the window.
Partition	A group of rows sharing the same PARTITION BY values; windows never cross partitions.
Frame	A sliding subset of the partition defined relative to the current row.
Peers	Rows with identical values in the window's ORDER BY columns.
ROWS mode	Frame boundaries counted in physical rows.
RANGE mode	Frame boundaries defined by values of the ordering column; peers are included together.
Running total	A cumulative sum from the first row of the partition up to the current row.
Moving average	The average over a fixed-size sliding frame of rows.
Tie-breaker	An additional ordering column that makes the order of otherwise equal rows unique.
CTE	Common table expression; a named temporary result defined at the start of a query.
Named window	A window specification defined in a WINDOW clause and reused by name.
Percentile	The value below which a given percentage of observations fall.
Bucket (tile)	One of the groups produced by NTILE.
Offset	The number of rows LAG or LEAD moves away from the current row.

Table 16.1 - Glossary of terms